KARKAII SCHOOL ERP · BACKEND · ARCH LINUX

Creating users and sessions with HTTPie

A hands-on walk-through you can run in a terminal and show to an audience. No frontend is involved: you talk to the API directly, so every step of sign-up, sign-in and session handling is visible.

Everything here was run against the real backend, and the output shown is what it returned (long values shortened). Total time: about 15 minutes.

What you will show

- **Create a user** and see exactly what comes back: a short-lived access token and a long-lived refresh cookie.
- **Sign in, create a school, switch into it**, then add a second person with a role (a teacher) and see that their powers are limited.
- **Keep sessions alive:** refresh a session, run two sessions for one person, log out.
- **Catch a stolen session:** replay an old cookie and watch the server shut the whole login down.

Sign up new account → **Sign in** token + cookie → **Create school** and switch into it → **Work** Bearer token → **Refresh** cookie, every 15 min → **Log out** session revoked

How a session works (read this once)

After sign-in the server gives you **two** tokens. They have different jobs:

	Access token	Refresh token
What it is	A signed JWT. It says *who* you are and *which school* you are acting in.	A random 64-character string. The server stores only a hash of it.
How it travels	In the JSON body (`data.accessToken`). You send it back as the header `Authorization: Bearer …`	In a cookie named `rt` : `HttpOnly` , path `/api/auth` . The client sends it back automatically.
Lifetime	15 minutes	30 days, but **single use**: each refresh swaps it for a new one
Used for	Every normal API call	Only `/auth/refresh` , `/auth/logout` , `/auth/switch-organization`

Two different things are called a “session”

- **An HTTPie session** (`--session=alice`) is a JSON file on your computer where HTTPie remembers cookies and the Authorization header between commands, like a browser’s cookie jar. One file per “browser” you want to play.
- **A server session** is the chain (“family”) of rows in the table `refresh_tokens` , one chain per sign-in. Logging out, or detecting theft, revokes the whole chain.

Background: where the data lives

Think of a hotel. The **access token** is the room keycard: it opens doors but stops working after 15 minutes. The **refresh token** is the pass that lets you get a new keycard at reception, and reception writes down every pass it hands out. The server is reception, and its only notebook is the database.

Three places, and what each one holds

Place	What it holds
The client a browser, or HTTPie's session file	The **access token** and the **refresh cookie** `rt`. In a real browser the cookie is hidden from JavaScript (`HttpOnly`).
The database table `refresh_tokens`	One row for every refresh token ever issued: the user, the school, a **hash** of the token (never the token itself), the family id, when it expires, when it was used, whether it was revoked, and the device (`user_agent`, `ip`).
The server's memory	**Nothing about sessions.** The server forgets everything between requests. There is no list of "logged-in users" to lose when it restarts.

The access token is **not stored anywhere on the server**. It is signed with a secret key. On each request the server checks its own signature on the token, then reads the user id (`sub`) and school id (`tid`) from it. Anyone can read a token; nobody can forge one without the key. The refresh token is kept only as a **hash**, like a fingerprint instead of the key, so a stolen copy of the database cannot be used to sign in.

What is stored at each moment

Moment	The client holds	Table `refresh_tokens`
Sign in	access token A1, cookie R1	new row for R1, in a new family F, unused
Normal request	sends A1 in the `Authorization` header	no change (the database is only used to look up the user's role)
Refresh A1 expired after 15 minutes	sends cookie R1; receives A2 and a new cookie R2	R1 marked **used**; new row R2 in the **same family F**
Log out	cookie deleted	every row of family F gets `revoked_at`, with `revoked_reason = logout`
Used cookie shown again after 10 seconds	an attacker sends R1	R1 is already used, so the whole family F is revoked, with `reuse_detected`

A refresh token can be exchanged **once**. That single rule is what makes theft visible: two people holding the same cookie cannot both use it, and the second attempt exposes them.

> **Permissions are not read from the token**
>
> The token only says *who* you are and *which school* you are in. On every request the server looks up your membership and roles in the database. This is why changing Bob's role, or suspending him, takes effect **immediately**, even while his old token is still inside its 15 minutes.

1. Prepare (once)

Install the two tools. HTTPie gives you the `http` command; `jq` picks the token out of a JSON reply.

terminal

```
sudo pacman -S --needed httpie jq
http --version
```

Start the backend (database, API, web pages). It asks for your sudo password once, for Docker. If you want to begin with an empty database, run `./demo.sh reset-db` first.

terminal

```
cd ~/karkaii
./demo.sh up
```

Now set three helpers for this terminal window. They keep the later commands short and are safe to paste into bash or zsh. (The demo password is fake; it needs at least 10 characters.)

terminal

```
export API=localhost:5000/api
export PW='Temp-pass-12345'
tok() { jq -r .data.accessToken; }
```

Check the backend answers:

terminal

```
http GET $API/health
```

you should see

```
HTTP/1.1 200 OK
...
{
    "data": { "status": "ok" },
    "success": true
}
```

A good screen layout

Window 1: the commands. Window 2: `./demo.sh logs backend`. Browser: `http://localhost:5555` (Prisma Studio), reloading the tables `users`, `memberships` and `refresh_tokens` as you go. The audience then sees every command land in the database.

Copying from this PDF

Commands contain no `#` comments, so they paste into zsh as well as bash. If a long line breaks when copied from the PDF, join it back into one line. Use the same email address throughout; if you repeat the whole demo, change the names or run `./demo.sh reset-db`, otherwise sign-up answers 409 (already exists).

2. The demo

1 Create a user (sign up)

Say: "One request creates the account and starts a session."

terminal

```
http --session=alice POST $API/auth/signup fullName="Alice Owner" email=alice@example.test password=$PW
```

response (trimmed)

```
HTTP/1.1 201 Created
Set-Cookie: rt=gO3kVn2...; Max-Age=2592000; Path=/api/auth; HttpOnly; SameSite=Lax

{
    "success": true,
    "data": {
        "accessToken": "eyJhbGciOiJIUzI1NiIs...",
        "expiresIn": 900,
        "user": { "id": "8", "name": "Alice Owner", "email": "alice@example.test",
                  "role": null, "permissions": [], "organization": null },
        "organizations": []
    }
}
```

Point out: the password is never returned; the refresh token is *only* in the cookie, never in the JSON; Alice has no school yet (`role: null` , `organizations: []`). A new account starts with no access to anything.

`--session=alice` told HTTPie to keep the cookie. See it:

terminal

```
cat ~/.config/httpie/sessions/localhost_5000/alice.json
```

response (trimmed)

```
"cookies": [
    {
        "name": "rt",
        "path": "/api/auth",
        "secure": false,
        "value": "gO3kVn2..."
    }
]
```

This file holds a live login. Do not paste it into a chat or a ticket. `secure` is false only because the demo runs on plain http; production will use https.

2 Sign in and keep the access token

Say: "Sign-in gives a new token. I store it in the HTTPie session so I do not have to paste it every time."

terminal

```
TOKEN=$(http --session=alice --print=b POST $API/auth/login email=alice@example.test password=$PW | tok)
http --session=alice -A bearer -a "$TOKEN" GET $API/auth/me
```

`--print=b` prints the body only, which `tok` reads. `-A bearer -a "$TOKEN"` sends `Authorization: Bearer …` **and HTTPie remembers it in the session**, so the following commands do not repeat it.

response (trimmed)

```
{
    "success": true,
    "data": {
        "name": "Alice Owner", "email": "alice@example.test",
        "role": null, "permissions": [],
        "organization": null, "organizations": []
    }
}
```

3 Create a school and switch into it

Say: "A person has one login but can belong to many schools. The active school is written inside the access token."

terminal

```
http --session=alice POST $API/organizations name="Green Valley School" slug=green-valley
```

response (trimmed)

```
HTTP/1.1 201 Created
{ "success": true,
  "data": { "id": "7", "name": "Green Valley School", "slug": "green-valley", "roles": ["owner"] } }
```

Alice is now the owner, but her *current token* still has no school. Ask for a token that does:

terminal

```
TOKEN=$(http --session=alice --print=b POST $API/auth/switch-organization organization=green-valley | tok)
http --session=alice -A bearer -a "$TOKEN" GET $API/auth/me
```

response (trimmed)

```
"role": "admin",
"permissions": [ "*" ],
"organization": { "slug": "green-valley", "name": "Green Valley School", "roles": [ "owner" ] }
```

Point out: `"*"` means every permission. The token carries the user id (`sub`) and the school id (`tid`). Anyone can read it, but nobody can forge it, because the server signs it with a secret key.

4 Create a second user with a limited role

Say: "The school owner adds a teacher. This creates the account *and* the membership in one call."

terminal

```
http --session=alice POST $API/members email=bob@example.test fullName="Bob Teacher" password=$PW
roles:='["teacher"]'
```

response (trimmed)

```
HTTP/1.1 201 Created
{ "success": true,
  "data": { "id": "12", "userId": "13", "name": "Bob Teacher", "email": "bob@example.test",
            "roles": [ "teacher" ], "status": "active" } }
```

`roles:=…` uses `:=` because the value is a JSON array, not a string. Now Bob signs in **in his own HTTPie session**, and we compare what he may do:

terminal

```
TB=$(http --session=bob --print=b POST $API/auth/login email=bob@example.test password=$PW | tok)
http --session=bob -A bearer -a "$TB" GET $API/members
http --session=bob GET $API/organizations/current
```

response (trimmed)

```
HTTP/1.1 403 Forbidden
{ "message": "You do not have permission to do this" }

HTTP/1.1 200 OK
{ "success": true, "data": { "name": "Green Valley School", "myRoles": [ "teacher" ], ... } }
```

Point out: Bob went straight into Green Valley as `teacher` because it is his only school. He may read the school but not list its members. Alice and Bob are two *separate* sessions, and the server treats them separately.

5 One person, a second device

Say: "Signing in again on a phone starts a second, independent session for the same person."

terminal

```
TP=$(http --session=alice-phone --print=b POST $API/auth/login email=alice@example.test password=$PW
organization=green-valley | tok)
http --session=alice-phone -A bearer -a "$TP" GET $API/auth/me
```

The optional `organization` field chooses which school to start in. Alice now has two independent logins (`alice` and `alice-phone`), each with its own cookie and its own chain in `refresh_tokens` .

6 Refresh: swap the cookie for a new token

Say: "The access token lives 15 minutes. When it expires the app quietly asks for a new one using the cookie, with no password."

terminal

```
SD=~/.config/httpie/sessions/localhost_5000
grep '"value"' $SD/alice.json
TOKEN=$(http --session=alice --print=b POST $API/auth/refresh | tok)
grep '"value"' $SD/alice.json
http --session=alice -A bearer -a "$TOKEN" GET $API/auth/me
```

Point out: the cookie value printed before and after is **different**. Each refresh token works exactly once; the reply carries a new cookie, and HTTPie saved it in the session file. Alice is still in Green Valley. The last line stores the new access token in the session. After 15 minutes the old one is rejected with `401` : that is the signal to refresh.

7 Log out

Say: "Logout kills the server-side session, not just the browser's copy."

terminal

```
http --session=alice-phone POST $API/auth/logout
http --session=alice-phone POST $API/auth/refresh
```

response (trimmed)

```
HTTP/1.1 204 No Content

HTTP/1.1 401 Unauthorized
{ "message": "Unauthorized" }
```

The phone is signed out, but the laptop session `alice` keeps working: each login is its own chain.

8 Catch a stolen session (the finale)

Say: "Suppose an attacker copies Alice's cookie. Watch what happens when both of them use it."

terminal

```
cp $SD/alice.json $SD/alice-stolen.json
http --session=alice POST $API/auth/refresh
http --session=alice-stolen POST $API/auth/refresh
```

response (trimmed)

```
HTTP/1.1 200 OK            the real user refreshes: fine, the cookie rotates
HTTP/1.1 409 Conflict       the old cookie again, within 10 seconds
{ "message": "Session refresh in progress" }
```

Say: "Within 10 seconds this looks like a harmless race, for example two browser tabs, so nothing is revoked." Now wait, then replay the old cookie again:

terminal

```
sleep 12
http --session=alice-stolen POST $API/auth/refresh
http --session=alice POST $API/auth/refresh
```

response (trimmed)

```
HTTP/1.1 401 Unauthorized     the old cookie, replayed late: this is theft
HTTP/1.1 401 Unauthorized     the REAL session is dead too
```

Point out: the server cannot tell who is the thief, so it revokes the **entire chain**. Both must sign in again (repeat step 2). In Prisma Studio, `refresh_tokens.revoked_reason` now says `reuse_detected` for that chain.

9 Show the errors, too

Without `--session`, these do not touch your saved logins.

terminal

```
http POST $API/auth/signup fullName="Alice Owner" email=alice@example.test password=$PW
http POST $API/auth/signup fullName="Zed Zed" email=zed@example.test password=short
http POST $API/auth/login email=alice@example.test password=wrong-password
http POST $API/auth/login email=nobody@example.test password=wrong-password
http GET $API/auth/me
```

Request	Answer	Message
sign up with an existing email	409	`An account with this email already exists`
sign up with a 5-character password	400	`Validation failed`, with `errors: [ { "field": "password", "message": "Password must be at least 10 characters" } ]`
wrong password *and* unknown email	401	the **same** message, `Invalid email or password`, so nobody can discover which emails exist
no token at all	401	`Unauthorized`

Sign-up and sign-in are limited to 10 attempts a minute per address; an eleventh within the minute answers `429`. Wait a minute and carry on.

10 Look at the database

Open `http://localhost:5555` (Prisma Studio) and show:

Table	What to point at
`users`	`password_hash` is an argon2 hash, never the password. `email` is stored in lower case. `last_login_at` updates at each sign-in.
`organizations`, `memberships`, `membership_roles`	Alice and Bob each have a membership in Green Valley; Alice's role is `owner`, Bob's is `teacher`.
`refresh_tokens`	`token_hash` (the raw cookie is never stored), `family_id` (one value per sign-in), `used_at` (set when a token is spent), `revoked_at` and `revoked_reason` (`logout` or `reuse_detected`), `user_agent` (shows `HTTPie/…`, the device that signed in).

The same view from a terminal:

```terminal
sudo docker exec karkaii-pg psql -U karkaii -d karkaii -c "select id, user_id, family_id, used_at is not null as used, revoked_reason from refresh_tokens order by id"
```

3. Reference

HTTPie in one table

You type	It means
`http POST $API/path`	send a POST (the method comes first; `http $API/path` alone is a GET)
`name=Alice`	a JSON **string** field. HTTPie sends JSON by default.
`roles:='["teacher"]'`	raw JSON: a number, `true`, `null`, an array or an object
`Header:value`	a request header
`q==bob`	a query string: `?q=bob`
`--session=NAME`	remember cookies and the auth header in a file called NAME
`-A bearer -a "$TOKEN"`	send `Authorization: Bearer …`
`--print=b` · `h` · `hb` · `H`	print the response body · response headers · both · the *request* headers
`-v`	show the full request *and* response
`--ignore-stdin`	needed in scripts (see below)

Show what HTTPie really sends, which makes the “two tokens” idea concrete:

terminal

```
http --session=bob --print=H GET $API/auth/me
```

response (trimmed)

```
GET /api/auth/me HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Cookie: rt=23bLMHwA5ctvJgN2...
User-Agent: HTTPie/3.2.4
```

When something goes wrong

You see	Cause and fix
`http: command not found` / `jq: command not found`	`sudo pacman -S --needed httpie jq`
`Connection refused`, or the API answers 500	The backend or its database is not running. `./demo.sh status`, then `./demo.sh up`. If Docker complains, check `sudo docker ps`.
`401 Unauthorized` on a normal call	(a) you never passed `-A bearer -a "$TOKEN"` in that session; (b) the token is older than 15 minutes: run the refresh in step 6; (c) the session was revoked by logout or theft detection: sign in again.
`401` on `/auth/refresh`	The cookie is missing, used, revoked or expired. Check you used the same `--session` name, then sign in again.
`409` on sign-up	That email already exists. Use another address or `./demo.sh reset-db`.
`409` on refresh	The same cookie was used twice within 10 seconds. Run it once.
`429 Too Many Requests`	Rate limit (10 sign-ups/sign-ins a minute). Wait one minute.
A command in a **script** answers 401 although the same command works by hand	When input is not a terminal, HTTPie assumes a request body is coming from stdin and silently changes GET into POST. In scripts, write the method (`http GET …`) and add `--ignore-stdin`.
zsh: `no matches found`	A `?` or `*` in a URL. Quote the URL, or use `q==bob` instead of `?q=bob`.

Start over

```terminal
rm -f ~/.config/httpie/sessions/localhost_5000/*.json
cd ~/karkaii && ./demo.sh reset-db
```

The first line forgets all saved logins; the second asks for confirmation and then empties the local database. Neither touches the production server.

Where this fits

These are the real endpoints the React app will call once it is connected: `/auth/signup`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `/auth/me` and `/auth/switch-organization`. The full list and the design are in `docs/BACKEND-OVERVIEW.pdf`; `./demo.sh api-demo` runs a narrated version of the same ideas with curl.